

Netflix Movie Recommendation System

Project Overview

This project builds a Netflix Movie Recommendation System using collaborative filtering, similarity-based recommendation techniques, and machine learning models. The goal is to recommend movies to users based on their viewing history and rating patterns.

Project Workflow

Step 1: Import Required Libraries

The project starts by importing the required Python libraries:

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-Learn
- SciPy
- Surprise Library

These libraries are used for:

- ✓ Data Processing
 - ✓ Visualization
 - ✓ Recommendation Modeling
 - ✓ Performance Evaluation
-

Step 2: Data Collection

Dataset Used:

Netflix Prize Dataset

Main Features:

Feature	Description
Movie ID	Unique movie identifier
User ID	Unique user identifier

Feature	Description
Rating	User rating (1-5)
Date	Rating date

Step 3: Data Preprocessing

Tasks Performed

- Merged multiple Netflix data files
- Removed missing values
- Removed duplicate records
- Converted date column into datetime format
- Sorted records based on date

Sample Data

Movie	User	Rating	Date
1	1488844	3	2005-09-06
1	822109	5	2005-05-13

Step 4: Exploratory Data Analysis (EDA)

Visualization 1: Distribution of Ratings

Objective

Understand how users rate movies.

Insight

- Most ratings are between 3 and 5.
- Users generally provide positive feedback.

Suggested Visualization:

Colorful Count Plot

```
sns.countplot(x='rating', data=train_df, palette='viridis')
```

Visualization 2: Ratings Over Time

Objective

Analyze user activity trends.

Insight

- Rating activity increases significantly over time.
- Platform engagement grows continuously.

Suggested Visualization:

Monthly Ratings Trend

```
train_df.resample('M', on='date')['rating'].count().plot()
```

Visualization 3: Number of Ratings Per User

Objective

Understand user engagement.

Insight

- Most users rate only a few movies.
- A small percentage of users are highly active.

Suggested Visualization:

KDE Distribution Plot

```
sns.kdeplot(no_of Rated movies per user,  
            fill=True,  
            color='purple')
```

Visualization 4: Quantile Analysis

Objective

Identify user-rating distribution.

Insight

- Rating behavior is highly skewed.
- Few users contribute most ratings.

Suggested Visualization:

Quantile Scatter Plot

```
plt.scatter(  
    quantiles.index,  
    quantiles.values,  
    color='orange'  
)
```

Visualization 5: Ratings Per Movie

Objective

Measure movie popularity.

Insight

- A small number of movies receive most ratings.
- Long-tail distribution exists.

Suggested Visualization:

Popularity Curve

```
plt.plot(no_of_ratings_per_movie.values)
```

Visualization 6: Ratings by Day of Week

Objective

Identify user activity patterns.

Insight

- Weekends generally receive higher engagement.
- Users are more active on leisure days.

Suggested Visualization:

Colorful Bar Chart

```
sns.countplot(  
    x='day_of_week',  
    data=train_df,  
    palette='coolwarm'  
)
```

Visualization 7: Box Plot of Ratings

Objective

Analyze rating spread across weekdays.

Insight

- Rating behavior remains relatively consistent.

Suggested Visualization:

Box Plot

```
sns.boxplot(  
    x='day_of_week',  
    y='rating',  
    data=train_df,  
    palette='Set3'  
)
```

Visualization 8: Average Rating Analysis

Objective

Compare average ratings.

Insight

- User average ratings vary.
- Movie average ratings remain more stable.

Suggested Visualization:

Dual KDE Plot

```
sns.kdeplot(avg_user_rating)  
sns.kdeplot(avg_movie_rating)
```

Step 5: Recommendation System Development

User-User Collaborative Filtering

Users with similar preferences are identified using:

- Cosine Similarity

Formula:

Similarity(User A, User B)

Higher similarity means stronger recommendation confidence.

Movie-Movie Collaborative Filtering

Movies are compared based on:

- Rating patterns
- User behavior

Benefits:

- ✓ Personalized recommendations
 - ✓ Improved recommendation quality
-

Step 6: Matrix Factorization (SVD)

Used:

- Truncated SVD
- SVD++
- Baseline Models

Purpose:

Reduce sparsity and discover hidden relationships between users and movies.

Visualization 9: Explained Variance by Latent Factors

Objective

Determine optimal latent dimensions.

Insight

- Variance increases with more latent factors.
- Diminishing returns after certain components.

Suggested Visualization:

SVD Explained Variance Plot

```
plt.plot(expl_var)
```

Step 7: Machine Learning Models

Models Trained:

1. Baseline Predictor
 2. KNN Baseline User
 3. KNN Baseline Movie
 4. SVD
 5. SVD++
 6. XGBoost Regressor
-

Step 8: Model Evaluation

Metrics Used

Metric	Purpose
RMSE	Error Measurement
MSE	Squared Error
MAE	Absolute Error

Lower values indicate better performance.

Results

Best Performing Models:

- SVD++
- XGBoost
- KNN Baseline

These models achieved the most accurate rating predictions.

Key Insights

- ✓ Users generally give positive ratings.
 - ✓ User activity increases over time.
 - ✓ Few movies dominate user attention.
 - ✓ Collaborative filtering effectively captures user preferences.
 - ✓ SVD improves recommendation quality significantly.
 - ✓ Ensemble machine learning models further enhance prediction accuracy.
-

Technologies Used

- Python
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-Learn
 - SciPy
 - Surprise
 - XGBoost
-

Future Improvements

- Deep Learning Recommenders
 - Neural Collaborative Filtering
 - Real-Time Recommendation Engine
 - Hybrid Recommendation System
 - Movie Metadata Integration
-

Conclusion

This project successfully develops a Netflix Movie Recommendation System using exploratory data analysis, collaborative filtering, matrix factorization, and machine learning.

The system demonstrates how user behavior and movie ratings can be leveraged to generate highly personalized movie recommendations.

